

# Python Stock Data Analysis

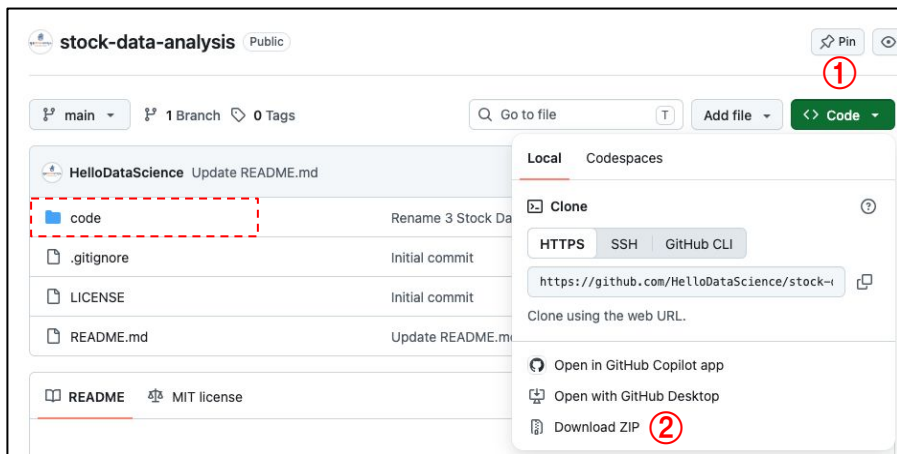
파이썬을 활용한 주식 데이터 분석

# 강의 내용

- 최소한의 Python 기초
- 주식 데이터의 이해
- 크롬 개발자 도구 활용법
- 웹 스크레이핑 실습
- 주식 데이터 수집 및 전처리
- 주식 데이터 분석 지표 계산
- 주식 데이터 시각화

# 실습 데이터셋 준비

- 이번 강의에서 사용할 실습 파일을 내려받을 수 있는 GitHub 저장소 링크를 안내합니다.
  - 링크: <https://github.com/HelloDataScience/stock-data-analysis>



① 링크에 접속하고 초록색 Code 버튼을 클릭합니다.

② Download ZIP을 클릭하여 압축 파일을 Downloads 폴더로 내려받습니다.

③ 탐색기를 열고 Downloads 폴더에서 압축 파일을 확인하고, 압축 파일을 해제합니다.

# [참고] code 폴더에는 완성된 Jupyter Notebook 파일이 포함되어 있습니다.

# 최소한의 Python 기초

# Python이란?

- Python은 네덜란드 출신의 컴퓨터 프로그래머인 귀도 반 로섬<sup>Guido van Rossum</sup>이 1991년에 발표한 프로그래밍 언어입니다. 귀도는 Google, Dropbox, Microsoft 등에서 활동했습니다.
- Python의 시작은 1989년 12월로 거슬러 올라갑니다. 당시 크리스마스 시즌이라 암스테르담의 연구소 사무실이 문을 닫자, 귀도는 집에 있는 컴퓨터로 이전부터 구상하던 새로운 스크립트 언어의 인터프리터를 작성하기 시작했고, 이것이 훗날 Python으로 발전했습니다.
- Python이라는 이름은 당시 귀도가 좋아하던 코미디 쇼인 "Monty Python's Flying Circus"에서 따온 것입니다.



- Python은 그리스 신화에 나오는 큰 뱀을 뜻하는 단어입니다.
- Python 버전은 2.x와 3.x가 있습니다. # [참고] Python 2.x는 공식 지원이 종료된 구버전이며, 데이터 분석 실무와 학습에서는 Python 3.x를 사용합니다.

# Python의 인기

- PYPL 지수 Popularity of Programming Language Index 기준  
현재 가장 인기 있는 언어는 Python입니다.
- PYPL 지수는 구글에서 각 프로그래밍 언어의  
튜토리얼이 얼마나 자주 검색되는지를 분석  
하여 만들어집니다.
  - 매월 전 세계 기준 결과를 공개합니다.
- Python은 범용 목적으로 개발된 언어이지만  
최근 데이터 분석과 머신러닝, 딥러닝 분야  
에서 많이 사용됩니다.

Worldwide, Jun 2026 :

| Rank | Change | Language    | Share   | 1-year trend |
|------|--------|-------------|---------|--------------|
| 1    |        | Python      | 45.23 % | +13.5 %      |
| 2    |        | Java        | 10.57 % | -4.6 %       |
| 3    | ↑      | C/C++       | 10.55 % | +3.6 %       |
| 4    | ↑↑     | R           | 5.01 %  | +0.1 %       |
| 5    | ↓↓     | JavaScript  | 4.23 %  | -3.5 %       |
| 6    | ↑↑↑↑   | Objective-C | 3.03 %  | +0.6 %       |
| 7    | ↑      | Rust        | 2.43 %  | -0.3 %       |
| 8    | ↓↓↓    | C#          | 2.37 %  | -3.4 %       |
| 9    | ↓↓     | PHP         | 2.36 %  | -1.2 %       |
| 10   | ↑      | Swift       | 2.35 %  | +0.2 %       |

# 출처: <https://pypl.github.io/PYPL.html>

# Python의 특징

- Python은 인간다운 언어이므로 사람이 생각하는 방식으로 프로그래밍합니다.
  - Python은 영어 문장처럼 읽히는 문법을 지향합니다.
- Python은 문법 구조가 단순하여 상대적으로 진입 장벽이 낮은 언어입니다.
  - 프로그래밍 사고에 익숙하지 않은 초심자는 문제를 해결하는 사고방식에 익숙해지는 데 상당한 시간이 필요합니다.
- Python은 오픈소스이므로 수많은 서드파티 라이브러리를 공짜로 사용할 수 있습니다.
  - 라이브러리 버전이 다르면 에러를 일으키고, 코딩 스타일에서 일관성이 부족합니다.
- Python은 간결한 코딩이 가능하지만, 들여쓰기 규칙을 준수해야 합니다.

# 출처: 점프 투 파이썬(<https://wikidocs.net/6>)에서 일부 발췌한 것입니다.

## [참고] Excel과 Python의 장단점 비교

| 구분 | Excel                                                                                                                                                                     | Python                                                                                                                                                                           |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 장점 | <ul style="list-style-type: none"> <li>- 복잡한 프로그래밍 지식이 필요 없음</li> <li>- 데이터를 시각적으로 표현하기 쉬움</li> <li>- 별도의 패키지를 설치하지 않아도 됨</li> <li>- 실수를 되돌릴 수 있음(undo)</li> </ul>        | <ul style="list-style-type: none"> <li>- 대량의 데이터를 효과적으로 처리 가능</li> <li>- 다양한 데이터 분석 및 머신러닝 가능</li> <li>- 반복 작업 및 자동화가 쉬움</li> <li>- 작업 과정을 코드로 남길 수 있어 재현 및 공유하기 쉬움</li> </ul>   |
| 단점 | <ul style="list-style-type: none"> <li>- 대량의 데이터를 처리하기 어려움</li> <li>- 복잡한 분석을 수행하는 데 제한이 있음</li> <li>- 반복적이고 복잡한 작업의 자동화가 어려움</li> <li>- 공동 작업에서 버전 관리를 하기 어려움</li> </ul> | <ul style="list-style-type: none"> <li>- 프로그래밍 지식이 필요함</li> <li>- 전체 데이터를 한눈에 보기 어려움</li> <li>- 초기 환경 설정이 필요하고 어려울 수 있음</li> <li>- 코드 실행 결과가 원본 객체에 반영되었다면 undo할 수 없음</li> </ul> |

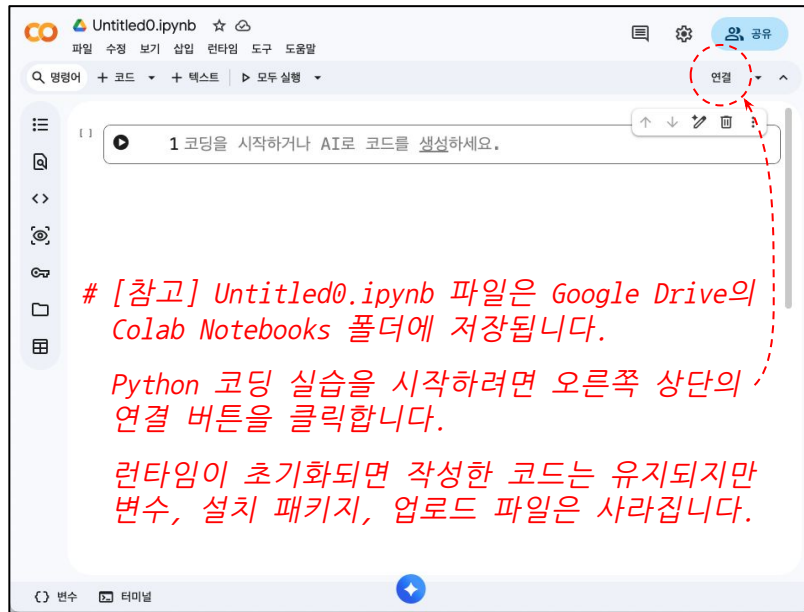
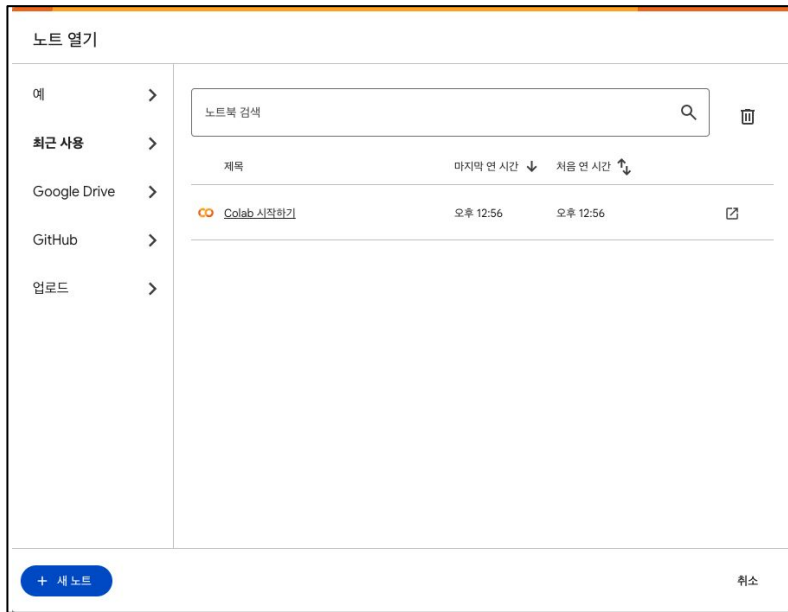
# [참고] Python 객체에 어떤 동작을 실행한 결과를 재할당하면 되돌릴 수 없으므로 미리 원본 객체를 복제하는 것이 좋을 수 있습니다.

# Colab이란?

- Colab은 Google이 웹 브라우저에서 Python 스크립트를 작성하고 실행할 수 있도록 개발한 온라인 에디터입니다.
  - Colab은 Colaboratory의 줄임말입니다.
- Google 계정이 있으면 Colab을 편리하게 이용할 수 있습니다.
  - Google에 로그인된 상태에서 다른 사람이 만든 Colab 파일을 복사하여 자신의 Google Drive에 사본으로 저장할 수 있습니다.
- 웹 브라우저에서 Colab을 검색하고 'Colab 시작하기'를 클릭하면 관련 메뉴가 열립니다.
  - 링크: <https://colab.research.google.com/?hl=ko>

## [참고] Colab 새 노트북 파일 열기

- 왼쪽 하단의 **+ 새 노트** 버튼을 클릭하면, 새 탭에서 Jupyter Notebook이 열립니다.



# Colab에서 Python 코딩 실습

- Colab에서 코드 또는 텍스트를 입력하는 부분을 셀<sup>cell</sup>이라고 합니다.
- 셀의 위/아래에 마우스를 가져다 놓으면 **+ 코드**와 **+ 텍스트** 버튼이 나타납니다.
  - **+ 코드** 버튼을 클릭하면 Python Code를 입력할 수 있는 셀을 추가합니다.
  - **+ 텍스트** 버튼을 클릭하면 Markdown, HTML tag 등 웹 문서 작성 셀을 추가합니다.



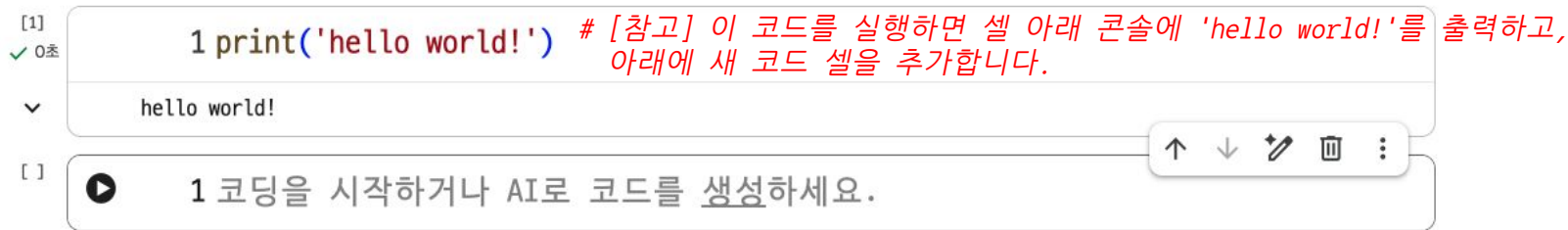
# 코드 실행 및 셀 추가 실습

- 현재 셀에 다음 코드를 입력합니다.

```
>>> print('hello world!')
```

# [참고] 해시(#) 뒤의 내용은 주석으로 처리합니다.  
주석은 일반적으로 코드 위 또는 오른쪽에 작성합니다.

- Python 코드 셀을 실행하는 두 가지 방법 중 하나를 선택하여 셀을 실행합니다.
  - 코드 셀 왼쪽에 있는 삼각형(▶) 버튼을 클릭합니다.
  - 코드 셀에서 shift + enter 키를 동시에 누릅니다.



# 셀 메뉴 소개

- 셀 오른쪽 상단에 있는 메뉴를 활용하면 셀에 대한 다양한 동작을 수행할 수 있습니다.

| 구분             | 상세 내용                                 |
|----------------|---------------------------------------|
| 셀 이동           | 위쪽 화살표 또는 아래쪽 화살표를 클릭합니다.             |
| 셀 삭제           | 휴지통을 클릭합니다.                           |
| 실행취소           | 수정 메뉴의 실행취소를 선택합니다.                   |
| 셀 선택, 복사, 잘라내기 | 삼점( :) 메뉴 버튼을 클릭하고 하위 메뉴를 선택합니다.      |
| 셀 붙여넣기         | ctrl + v 키를 동시에 누르면 커서 아래에 셀을 붙여넣습니다. |

[ ]



1 코딩을 시작하거나 AI로 코드를 생성하세요.

+ 코드

+ 텍스트



# 작업 경로 확인 및 변경

- 필요한 모듈을 임포트합니다.

```
>>> import os
```

- 현재 작업 경로를 확인합니다.

```
>>> os.getcwd() # [참고] Colab에서 기본 작업 경로는 /content 폴더입니다.
```

- sample\_data 폴더로 작업 경로를 변경합니다.

```
>>> os.chdir(path='/content/sample_data')
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> sorted(os.listdir()) # [참고] sample_data 폴더에는 Colab에서 기본으로 제공하는 여섯 개의 예제 파일이  
저장되어 있으며, 파일 구성은 Colab 환경에 따라 달라질 수 있습니다.
```

# csv 파일 읽고 데이터프레임 생성

- 필요한 모듈을 임포트합니다.

```
>>> import pandas as pd
```

- 읽을 csv 파일명을 file\_name에 할당합니다.

```
>>> file_name = 'california_housing_train.csv'
```

- csv 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df = pd.read_csv(filepath_or_buffer=file_name)
```

- df의 처음 5행을 확인합니다.

```
>>> df.head() # [참고] head 메서드의 n 매개변수에 지정한 정수만큼 행을 반환합니다.(기본값: 5)
```

# 데이터프레임을 csv 파일로 저장

- df를 'train.csv' 파일로 저장합니다.

```
>>> df.to_csv(path_or_buf='train.csv', index=False)
```

- 현재 작업 경로에 있는 원본 csv 파일을 삭제합니다.

```
>>> os.remove(path=file_name)
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> sorted(os.listdir())
```

- 상단 메뉴에서 런타임 -> 런타임 연결 해제 및 삭제를 클릭한 뒤, 다시 연결을 실행하면 sample\_data 폴더가 초기화됩니다.

# 구글 드라이브 마운트

- 필요한 모듈을 임포트합니다.

```
>>> from google.colab import drive
```

- 구글 드라이브에 연결합니다.

```
>>> drive.mount(mountpoint='/content/drive')
```

- 오른쪽 팝업이 열리면 'Google Drive에 연결' 버튼을 클릭합니다.
  - 구글 계정을 선택하고, 구글 드라이브에 연결할 항목을 선택합니다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

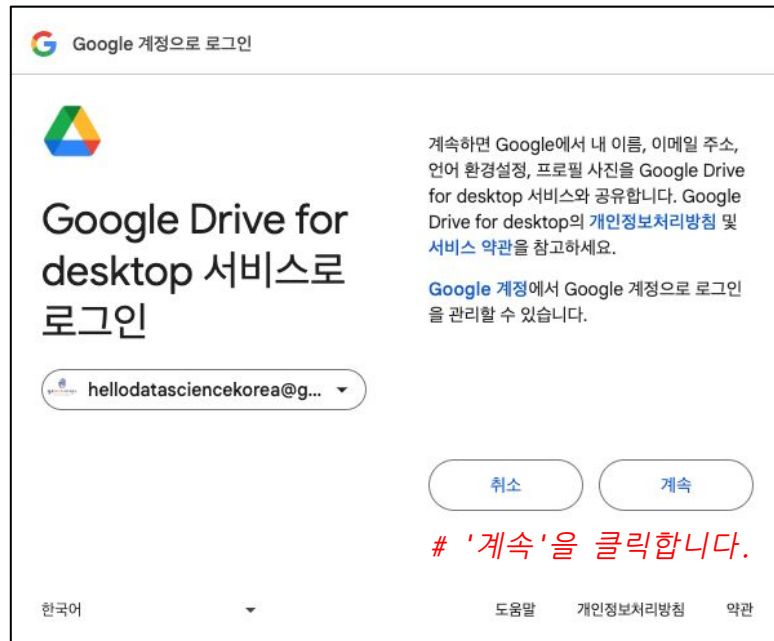
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

Google Drive에 연결

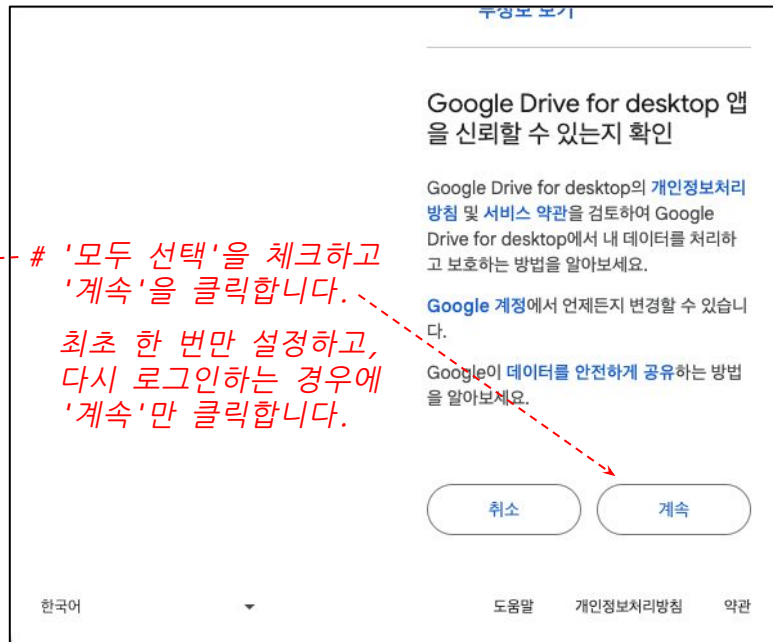
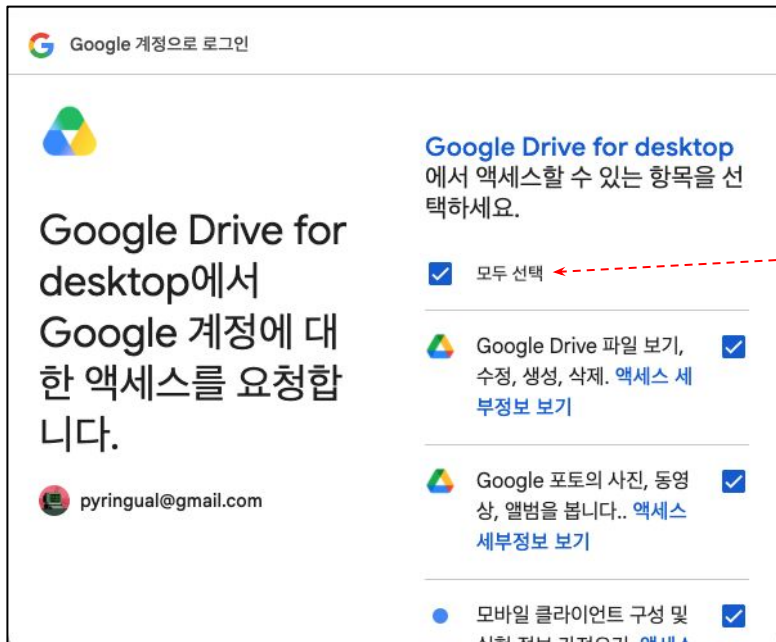
# 구글 드라이브 마운트(계속)

- 구글 드라이브에 연결할 계정을 선택하고, 로그인합니다.



# 구글 드라이브 마운트(계속)

- Google 계정에 대한 액세스 요청 항목을 선택합니다.



# 구글 드라이브에 새 폴더 생성

- 필요한 모듈을 임포트합니다.

```
>>> import os # [참고] 런타임을 초기화하면 필요한 모듈을 다시 임포트해야 합니다.
```

- 새 폴더 경로를 folder\_path에 할당합니다.

```
>>> folder_path = '/content/drive/MyDrive/workspace' # [참고] '/content/drive/MyDrive'는 고정이고  
                                                    'workspace' 부분은 변경할 수 있습니다.
```

- 구글 드라이브의 내 드라이브 폴더에 새 폴더를 생성합니다.

```
>>> os.makedirs(name=folder_path, exist_ok=True) # [참고] 실습 파일을 새로 만든 폴더에 업로드한 뒤  
                                                    마지막 코드를 실행합니다.
```

- folder\_path에 있는 폴더명과 파일명을 확인합니다.

```
>>> sorted(os.listdir(path=folder_path)) # [참고] MyDrive에 실습 파일을 저장하면 런타임을 초기화해도  
                                         계속 사용할 수 있습니다.
```

## [참고] Python 기초 문법

- 데이터 분석을 위한 Python 기초 문법은 다음 내용을 주로 다룹니다.
  - 변수와 객체: 객체 지향 프로그래밍의 특징
  - 단순 자료형: 정수, 실수, 문자열, 부울
  - 복합 자료형: 리스트, 튜플, 집합, 딕셔너리
  - 조건문: if, 삼항 연산자
  - 반복문: for, while, 리스트 컴프리헨션
  - 함수: def, lambda
  - 클래스: class

# 주식 데이터의 이해

# 주식 데이터의 구조

- 주식 데이터는 시간의 흐름에 따라 기록되는 대표적인 시계열 데이터입니다.
  - 각 시점마다 주식 가격과 거래량 정보가 함께 기록됩니다.
- 주식 데이터는 날짜 인덱스와 여러 수치형 변수로 구성된 데이터프레임의 형태로 표현됩니다.
  - 주식 데이터는 날짜를 기준으로 정렬되며, 각 날짜마다 동일한 구조를 갖습니다.
  - 다섯 가지 수치형 변수는 주가의 움직임과 거래 강도를 설명하는 핵심 지표입니다.

| Datetime | Open | High | Low | Close | Volume |
|----------|------|------|-----|-------|--------|
| 거래일시     | 시가   | 최고가  | 최저가 | 종가    | 거래량    |

## [참고] 시계열 데이터 vs 횡단면 데이터

- 시계열 데이터란 시간의 흐름에 따라 순차적으로 관측한 데이터입니다.
  - 개별 관측값은 특정 시점에 기록되며, 관측값 사이의 순서는 단순한 정렬 정보가 아니라 데이터가 어떤 현상을 설명하는지를 결정하는 핵심 요소입니다.
  - 대표적인 시계열 데이터는 주식, 매출, 기온, 방문자 수 등이 있습니다.
- 횡단면 데이터는 동일한 시점에서 여러 개체를 동시에 관측한 데이터입니다.
  - 횡단면 데이터는 샘플의 순서를 섞어도 분석 결과에 큰 영향을 주지 않는 경우가 많지만 시계열 데이터에서는 이러한 접근이 허용되지 않습니다.
  - 시계열 데이터의 관측값 순서를 변경하면 원래의 현상을 반영하지 못하게 됩니다.

## [참고] 시계열 데이터 vs 횡단면 데이터

|            | 대여량 | 기온(℃) | 습도(%) | 요일  |
|------------|-----|-------|-------|-----|
| 2020-01-01 | 128 | 3.2   | 72    | 수요일 |
| 2020-01-02 | 121 | 4.8   | 67    | 목요일 |
| 2020-01-03 | 157 | 6.4   | 58    | 금요일 |
| 2020-01-04 | 194 | 8.2   | 52    | 토요일 |
| 2020-01-05 | 255 | 9.6   | 47    | 일요일 |
| ⋮          | ⋮   | ⋮     | ⋮     | ⋮   |

횡단면 데이터

# [참고] 횡단면 데이터는 동일한 시점에서 여러 개체를 동시에 관측한 데이터입니다.

시계열 데이터

# [참고] 시계열 데이터는 시간의 흐름에 따라 순차적으로 관측한 데이터입니다.

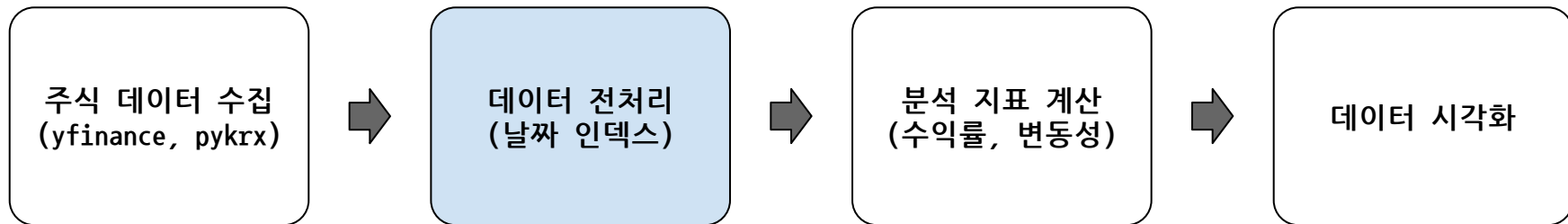
# 주식 데이터의 구성 요소

- 주식 데이터는 다섯 가지 핵심 변수로 구성됩니다.
  - Open(시가): 장이 시작될 때 형성된 첫 거래 가격입니다. # [참고] 시가는 장 시작 전 동시호가로 결정됩니다.
  - High(고가): 해당 날짜 동안 거래된 가격 중 가장 높은 값입니다.
  - Low(저가): 해당 날짜 동안 거래된 가격 중 가장 낮은 값입니다.
  - Close(종가): 장이 종료될 때 마지막으로 거래된 가격입니다.
    - 종가는 시장 참여자들의 합의된 최종 가격이므로 데이터 분석에서 가장 많이 사용됩니다.
  - Volume(거래량): 해당 날짜 동안 거래된 주식의 총 수량입니다.
    - 거래량은 주가 움직임의 신뢰도를 판단하는 기준으로 활용됩니다.

# 날짜 인덱스의 중요성

- 주식 데이터에서 인덱스는 단순한 번호가 아니라 시간 정보를 의미합니다.
  - 특정 기간을 선택하거나 이동 평균을 계산할 때 날짜 인덱스가 중요한 역할을 합니다.
- 주식 데이터를 다음 과정을 통해 전처리합니다.
  - 거래일을 날짜 인덱스로 설정합니다.
  - 날짜 인덱스를 기준으로 데이터를 오름차순 정렬합니다.
- 데이터프레임에서 날짜시간형 시리즈를 인덱스로 설정하면, 데이터프레임의 인덱스 자료형은 `DatetimeIndex`로 변환됩니다.
  - `DatetimeIndex`에는 날짜 필터링, 리샘플링, 날짜 요소 추출 등을 쉽게 처리하는 장점이 있습니다.

# 주식 데이터 분석 프로세스



# 크롬 개발자 도구 활용법

네트워크 및 요소 탭

# 크롬 개발자 도구를 사용하는 이유

- 크롬 개발자 도구는 다음 두 가지 문제를 해결하는 데 유용합니다.
  - 네트워크<sup>Network</sup> 탭: HTTP 요청 및 응답 리소스를 찾을 수 있습니다.
  - 요소<sup>Elements</sup> 탭: 수집하려는 텍스트를 포함한 HTML 요소를 찾을 수 있습니다.
- 크롬 개발자 도구를 여는 방법은 다음과 같습니다.
  - 방법 1: 크롬 브라우저 오른쪽 상단에 있는 삼점 메뉴( : ) 클릭 → 도구 더보기<sup>More Tools</sup> → 개발자 도구<sup>Developer Tools</sup>를 차례대로 선택합니다.
  - 방법 2: 크롬 브라우저에서 마우스 오른쪽 버튼을 클릭하면 팝업 메뉴가 열리고, 팝업 메뉴에서 검사<sup>Inspect</sup>를 선택합니다.  
# [참고] Windows는 F12 키를 지원합니다.
  - 방법 3: 단축키를 사용합니다. Windows: ctrl + shift + I, MacOS: cmd + opt + I

# HTTP 통신 리소스 찾기

- 타겟 페이지로 이동하여 수집할 데이터를 확인하고 크롬 개발자 도구를 엽니다.
  - 타겟 페이지: <https://finance.naver.com/item/main.naver?code=005930>

The screenshot shows the Chrome DevTools Network tab. The 'Network' tab is selected, and the 'All' filter is active. A red dashed box highlights the filter buttons: 'All', 'Fetch/XHR', 'Doc', 'CSS', 'JS', 'Font', 'Img', 'Media', 'Manifest', 'WS', 'Wasm', and 'Other'. Another red dashed box highlights the table headers: 'Name', 'Status', 'Type', 'Initiator', 'Size', and 'Time'.

# All: 모든 리소스를 포함하고 있습니다.  
 Doc: 정적인 HTML 리소스를 여기에서 찾습니다.  
 Fetch/XHR: 비동기 HTTP 요청 리소스를 여기에서 찾습니다.

| Name                                                                             | Status | Type | Initiator | Size | Time |
|----------------------------------------------------------------------------------|--------|------|-----------|------|------|
| # Name: 리소스 목록을 확인합니다.<br>Status: 리소스의 응답 상태 코드를 확인합니다.<br>Type: 리소스의 종류를 확인합니다. |        |      |           |      |      |

# HTTP 통신 리소스 찾기(계속)

- 삭제 버튼을 클릭하고 새로고침하면 HTTP 요청을 실행하고 리소스를 표시합니다.

Network tab showing a request to `main.naver?code=005930`. The 'Preview' tab is selected, displaying the response body in Korean.

종목 시세 정보  
 2024년 05월 23일 16시 11분 기준 장마감  
 종목명 삼성전자  
 종목코드 005930 코스피  
 현재가 78,300 전일대비 상승 600 플러스 0.77 퍼센트  
 전일가 77,700  
 시가 77,700  
 고가 79,100  
 상한가 101,000  
 저가 77,100  
 하한가 54,400  
 거래량 18,595,182  
 거래대금 1,451,971백만

# [참고] 크롬 버전 문제로 문자 인코딩 방식이 UTF-8이 아닌 한글을 제대로 출력하지 못할 수 있습니다.

# [참고] 'main'으로 시작하는 리소스를 클릭하고 Preview로 이동하면, 해당 리소스의 HTTP 응답 바디 문자열(HTML)을 웹 페이지에 표시한 결과를 미리보기할 수 있습니다.

# HTTP 통신 리소스 찾기(계속)

- 리소스의 Headers로 이동하여 요청 방식과 요청 URL을 확인합니다.

The screenshot shows the Chrome DevTools Network tab. The 'Headers' panel is open, displaying the following information:

| General           |                                                       |
|-------------------|-------------------------------------------------------|
| Request URL:      | https://finance.naver.com/item/main.naver?code=005930 |
| Request Method:   | GET                                                   |
| Status Code:      | 200 OK                                                |
| Remote Address:   | 117.52.137.138:443                                    |
| Referrer Policy:  | strict-origin-when-cross-origin                       |
| Response Headers  |                                                       |
| Cache-Control:    | no-cache                                              |
| Content-Encoding: | gzip                                                  |
| Content-Language: | ko                                                    |
| Content-Type:     | text/html; charset=EUC-KR                             |
| Date:             | Fri, 05 Jan 2024 02:10:00 GMT                         |
| Expires:          | Thu, 01 Jan 1970 00:00:00 GMT                         |
| Referrer-Policy:  | unsafe-url                                            |
| Server:           | nfront                                                |

# HTML 요소 찾기

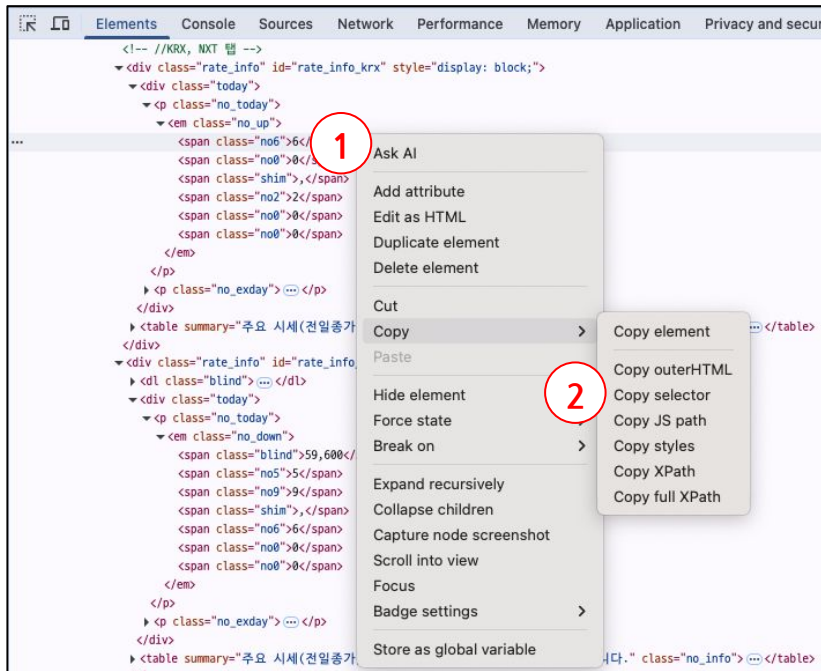
- 선택 버튼을 한 번 클릭하고(파란색) 웹 페이지에서 수집할 텍스트를 선택합니다.

# 웹 페이지에서 마우스가 가리키는 HTML 요소를 파란색 블록으로 표시합니다.  
[참고] 마우스를 클릭하면 해당 블록을 고정시킵니다.

# 크롬 개발자 도구에서 파란색 블록에 해당하는 HTML 요소를 회색 블록으로 표시하며  
왼쪽 □를 클릭하면 □로 바뀌면서 해당 HTML 요소의 자식 요소를 펼쳐서 표시합니다.

# HTML 요소의 CSS 선택자 복사

- 다음 과정을 거쳐 HTML 요소의 CSS 선택자를 복사합니다.



- ① HTML 요소 위에 마우스 오른쪽 버튼을 클릭합니다.
- ② 팝업 메뉴에서 Copy → Copy selector를 선택합니다.
- ③ Jupyter Notebook에 복사한 값을 붙여넣습니다.

3

```
[ ]: items = soup.select(selector = '#rate_info_krx > div > p.no_today > em > span.no6')
len(items)
```

# 웹 스크레이핑 실습

현재 주식 가격 수집

# 현재 주식 가격 페이지 탐색

- 네이버 증권(<https://finance.naver.com/>) 페이지로 이동합니다.
- 검색창에 종목명을 입력하고 관련 페이지로 이동합니다.(예: 삼성전자)
- 종목명 오른쪽에 있는 6자리 종목코드를 확인합니다.(예: 005930)
- 크롬 개발자 도구를 열고 네트워크 탭에서 Doc로 이동합니다.
- 삭제 버튼을 클릭하고 웹 페이지를 새로고침합니다.
- 'main.naver'로 시작하는 리소스를 클릭합니다.
- Preview에서 수집하려는 텍스트를 찾습니다.
- Headers로 이동하여 HTTP 요청 방식을 확인하고 요청 URL을 복사합니다.

# HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 상장회사의 종목코드를 설정합니다.

```
>>> ticker = '005930' # [참고] '005930'은 삼성전자 보통주의 종목코드입니다.
```

- 요청 URL을 설정합니다. # [참고] 크롬 개발자 도구 리소스의 Headers에서 요청 URL을 복사하여 붙입니다.  
마지막 여섯 자리 숫자는 종목코드이므로 f-문자열로 대체하는 것이 좋습니다.

```
>>> url = f'https://finance.naver.com/item/main.naver?code={ticker}'
```

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url) # [참고] verify 매개변수에 False를 지정하면 SSL 인증서 검증을 비활성  
하므로(기본값: True) 원인 확인 없이 사용하는 것은 권장하지 않습니다.
```

# HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code # [참고] 상태 코드가 200이면 다음 코드를 실행할 수 있습니다.
```

- HTTP 응답 헤더에서 콘텐츠 유형을 확인합니다.

```
>>> res.headers['content-type'] # [참고] 응답 바디의 콘텐츠 유형은 'text/html; charset=EUC-KR'입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text) # [참고] 응답 바디 문자열을 print 함수로 출력하는 것이 보기에 좋습니다.
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url) # [참고] 만약 응답 바디 문자열에 수집하려는 텍스트가 없으면 요청 URL을 웹 브라우저에 붙여넣고 실행했을 때 웹 페이지가 제대로 열리면 HTTP 요청 코드를 수정해야 합니다.
```

# HTML 데이터 처리

- 필요한 모듈을 임포트합니다.

```
>>> from bs4 import BeautifulSoup as BTS
```

- HTTP 응답 바디 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

```
>>> soup = BTS(markup=res.text, features='html.parser')
```

# [참고] features 매개변수를 생략하면  
'lxml', 'html5lib', 'html.parser'(내장)  
순으로 자동 선택합니다.

- soup을 확인합니다.

```
>>> soup
```

- soup을 HTML의 계층 구조(들여쓰기)를 반영한 문자열로 변환하여 출력합니다.

```
>>> print(soup.prettify())
```

# [참고] prettify 메서드는 BeautifulSoup 객체를 HTML의 계층 구조에 맞게 포맷팅  
하여 가독성 좋은 문자열로 반환합니다.

# 현재 주식 가격 수집

- 현재 주식 가격 정보를 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector='#rate_info_krx > div > p.no_today > em > span')
```

- items의 원소 개수를 확인합니다.

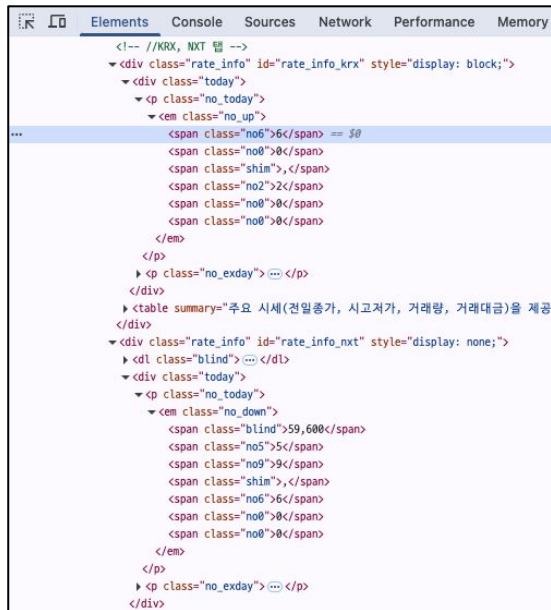
```
>>> len(items)
```

- items를 확인합니다.

```
>>> items # [참고] items는 리스트 형태의 자료형이며 첫 번째 원소에  
# 수집하려는 텍스트가 포함되어 있습니다.
```

- items의 첫 번째 원소에서 문자열을 추출합니다.

```
>>> items[0].text # [참고] items의 첫 번째 원소에서 시작 태그와  
# 종료 태그 사이에 있는 문자열을 선택합니다.
```



```
<!-- //KRX, NXT 탭 -->
<div class="rate_info" id="rate_info_krx" style="display: block;">
  <div class="today">
    <p class="no_today">
      <em class="no_up">
        <span class="no6">59,600</span>
        <span class="no0">0</span>
        <span class="shim"></span>
        <span class="no2">2</span>
        <span class="no0">0</span>
        <span class="no0">0</span>
      </em>
    </p>
    <p class="no_exday">...</p>
  </div>
  <table summary="주요 시세(전일종가, 시고저가, 거래량, 거래대금)을 제공">
  </table>
  <div class="rate_info" id="rate_info_nxt" style="display: none;">
    <dl class="blind">...</dl>
    <div class="today">
      <p class="no_today">
        <em class="no_down">
          <span class="blind">59,600</span>
          <span class="no5">5</span>
          <span class="no9">9</span>
          <span class="shim"></span>
          <span class="no6">6</span>
          <span class="no0">0</span>
          <span class="no0">0</span>
        </em>
      </p>
      <p class="no_exday">...</p>
    </div>
  </div>
```

# 현재 주식 가격 수집 함수 생성

- 종목코드를 지정하면 현재 주식 가격을 반환하는 함수를 생성합니다.

```
>>> def stock_price(ticker):  
  
    url = f'https://finance.naver.com/item/main.naver?code={ticker}'  
  
    res = requests.get(url=url)  
  
    soup = BeautifulSoup(markup=res.text, features='html.parser')  
  
    items = soup.select(selector='#rate_info_krx > div > p.no_today > em > span')  
  
    return items[0].text
```

# [참고] 사용자 정의 함수 내부에 없는 모듈(requests)과 클래스(BTS)는 전역 변수 목록에서 참고합니다.

[주의] 만약 전역 변수 목록에 해당 모듈과 클래스가 없으면 사용자 정의 함수를 실행했을 때 에러를 일으킵니다.

# 여러 종목의 현재 주식 가격 수집

- 관심 있는 종목코드를 리스트로 생성합니다.

```
>>> tickers = ['005930', '000660', '402340', '005935', '009150']
```

- 관심 있는 종목코드의 종목명을 리스트로 생성합니다.

```
>>> cpnames = ['삼성전자', '하이닉스', 'SK스퀘어', '삼전우선', '삼성전기']
```

- 반복문으로 종목별 현재 주식 가격을 수집하고 출력합니다.

```
>>> for ticker, cpname in zip(tickers, cpnames):
```

```
    price = stock_price(ticker=ticker)
```

```
    print(f'{cpname}의 현재 주식 가격은 {price}원입니다.') # [참고] 마지막 행에 time.sleep  
                                                           함수를 추가하는 것이 좋습니다.
```

# 주식 데이터 수집 및 전처리

yfinance 라이브러리 사용

# yfinance 라이브러리 소개

- yfinance는 Yahoo Finance의 공개 데이터를 기반으로 주식, ETF, 지수 등의 시계열 데이터를 손쉽게 수집할 수 있도록 지원하는 Python 라이브러리입니다.
  - 주식 가격 데이터(Open, High, Low, Close, Volume)와 일부 재무 데이터를 데이터프레임 형태로 불러올 수 있어, 데이터 분석에서 널리 활용됩니다.
  - 별도의 인증키 없이도 글로벌 주식 데이터를 즉시 수집할 수 있다는 장점이 있습니다.
  - 웹 데이터 기반으로 제공하는 서비스이므로 데이터의 안정성과 지속성이 보장되지 않고, 호출 제한이나 구조 변경에 영향을 받을 수 있습니다.
  - 미국 주식 데이터는 안정성과 정확도가 높은 편이지만 국내 주식 데이터는 데이터 누락, 수정 종가 미반영, 가격 불일치 등의 문제가 발생할 수 있습니다.

## [참고] 수정 종가

- 배당, 액면분할, 무상증자 등의 기업 이벤트가 발생하면 주가는 변동됩니다.

| 구분        | 상세 내용                                    |
|-----------|------------------------------------------|
| 배당        | 배당락일에 배당금에 해당하는 만큼 기준가격이 조정됩니다.          |
| 액면분할/무상증자 | 주식 수가 증가하므로 분할 또는 증자 비율에 따라 기준가격이 조정됩니다. |

- 수정 종가는 기업 이벤트의 영향을 반영하여 과거 가격을 현재 기준으로 보정한 가격입니다.
  - 장기 수익률 분석이나 백테스트에서는 수정 종가를 사용하는 것이 일반적입니다.
  - 데이터 제공처나 종목에 따라 수정 종가가 제공되지 않거나 조정 방식에 차이가 있을 수 있습니다.

# [참고] 이번 실습에서는 배당을 포함한 총수익률이 아니라 종가 기준의 가격 수익률을 분석합니다.

# 필요한 모듈 импорт

- Colab에 yfinance 라이브러리 설치 여부를 확인합니다.

```
>>> !pip show yfinance
```

# [참고] pip show는 설치된 라이브러리의 정보를 콘솔에 출력합니다.  
만약 해당 라이브러리가 설치되지 않았으면 Package(s) not found 경고를 출력합니다.

- yfinance 라이브러리를 설치합니다.

```
>>> !pip install yfinance
```

# [참고] 위 코드를 실행한 결과 Name, Version 등의 패키지 정보가 출력되었으면  
설치 코드는 생략합니다.

- 필요한 모듈을 импорт합니다.

```
>>> import numpy as np
```

```
>>> import pandas as pd
```

```
>>> import yfinance as yf
```

# 주식 데이터 수집

- 티커(종목코드)를 설정합니다.

```
>>> ticker = '005930.KS' # [참고] 유가증권시장(KOSPI) 종목은 여섯 자리 종목코드 뒤에 '.KS'를 추가합니다.  
                        (코스닥은 '.KQ')
```

- 주식 데이터 수집 시작일자를 설정합니다.

```
>>> start_date = '2020-01-01' # [참고] 단순 수익률을 계산할 때 첫 번째 행은 결측이 됩니다.
```

- 주식 데이터 수집 종료일자를 설정합니다.

```
>>> end_date = pd.Timestamp.today().strftime(format='%Y-%m-%d') # [참고] 현재 시스템 날짜시간을  
                                                                'yyyy-mm-dd' 형태의 문자열로  
                                                                변환해 end_date에 할당합니다.
```

- end\_date를 확인합니다.

```
>>> end_date
```

## 주식 데이터 수집(계속)

- 주식 데이터를 수집하고 데이터프레임을 생성합니다.

```
>>> df = yf.download(tickers=ticker,  
  
                      start=start_date,  
  
                      end=end_date, # [참고] end 매개변수에 지정한 날짜를 포함하지 않습니다.  
  
                      auto_adjust=False, # [참고] auto_adjust 매개변수에 False를 지정하면 조정되지 않은  
                                         OHLC를 유지하고 수정 종가를 함께 반환합니다.(기본값: True)  
  
                      progress=False, # [참고] progress 매개변수에 False를 지정하면 다운로드 진행 표시를  
                                       출력하지 않습니다.(기본값: True)  
  
                      multi_level_index=False) # [참고] multi_level_index 매개변수에 False를 지정하면  
                                                단일 레벨 열 이름을 반환합니다.(기본값: True)
```

# 데이터프레임 확인

- df의 처음 5행을 확인합니다.

```
>>> df.head() # [참고] n 매개변수에 출력할 행 개수를 정수로 지정합니다.(기본값: 5)
```

- df의 컬럼스(열 이름)를 확인합니다.

```
>>> df.columns # [참고] 열 이름은 Adj Close(수정 종가), Close(종가), High(고가), Low(저가), Open(시가),  
               Volume(거래량) 순입니다.
```

- df의 인덱스(행 이름)를 확인합니다.

```
>>> df.index # [참고] 인덱스 자료형은 DatetimeIndex입니다.
```

- df의 정보를 확인합니다.

```
>>> df.info() # [참고] 행 개수, 열 개수, 열 이름, 열별 결측값 아닌 개수 및 열별 자료형을 출력합니다.
```

## 열 선택 및 순서 변경

- df에서 Adj Close 열을 삭제합니다.

```
>>> df = df.drop(columns='Adj Close')
```

- df의 열 순서를 변경하기 위해 열 이름 리스트를 생성합니다.

```
>>> cols = ['Open', 'High', 'Low', 'Close', 'Volume']
```

- df의 열 순서를 변경합니다.

```
>>> df = df.loc[:, cols]
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

# 지저분한 데이터 만들기

- df의 인덱스를 초기화하고 기존 인덱스를 첫 번째 열로 삽입합니다.

```
>>> df = df.reset_index() # [참고] drop 매개변수에 True를 지정하면 기존 인덱스를 삭제합니다.  
                        (기본값: False)
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

- df에서 Date를 문자열로 변환합니다.

```
>>> df['Date'] = df['Date'].astype(dtype=str)
```

- df의 열별 자료형을 확인합니다.

```
>>> df.dtypes # [참고] Date의 자료형이 object입니다.
```

## 지저분한 데이터 만들기(계속)

- df의 행 순서를 무작위로 섞어서 시간 순서를 망가뜨립니다.

```
>>> df = df.sample(frac=1, random_state=1) # [참고] 매번 같은 결과를 얻기 위해 시드를 고정합니다.
```

- df의 처음 5행을 확인합니다.

```
>>> df.head() # [참고] df의 인덱스가 정수 0부터 시작하지 않고, Date가 무작위로 섞여 있습니다.
```

- df의 인덱스를 초기화하고, 기존 인덱스를 삭제합니다.

```
>>> df = df.reset_index(drop=True)
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

# 날짜 인덱스 전처리

- df에서 Date를 날짜시간형으로 변환합니다.

```
>>> df['Date'] = pd.to_datetime(arg=df['Date'])
```

- df에서 Date를 인덱스로 설정합니다.

```
>>> df = df.set_index(keys='Date') # [참고] 시계열 데이터는 시간 순서가 중요하므로 날짜시간 데이터를  
# 인덱스로 설정하면 기간 선택 및 이동 통계량 계산이 편리해집니다.
```

- df를 인덱스 기준으로 오름차순 정렬합니다.

```
>>> df = df.sort_index() # [참고] 날짜 인덱스로 오름차순 정렬해야 단순 수익률 및 이동 통계량 등을 계산할  
# 수 있습니다.
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

## [참고] 누락된 인덱스 추가 및 결측값 처리

- df의 인덱스를 일(day) 단위로 재생성하고, 처음 5행만 선택하여 df\_na에 할당합니다.

```
>>> df_na = df.asfreq(freq='D').head()
```

```
>>> df_na # df_na를 확인합니다.
```

*# [주의] 아래 코드는 결측값 처리 메서드의 사용법을 확인하기 위한 예시이며, 실제 비거래일의 거래량을 채우는 용도로 사용하지 않습니다.*

- df\_na에서 거래량의 결측값을 직전 값으로 채운 결과를 확인합니다.

```
>>> df_na['Volume'].ffill()
```

- df\_na에서 거래량의 결측값을 선형 보간법으로 채운 결과를 확인합니다.

```
>>> df_na['Volume'].interpolate(method='linear')
```

## [참고] 전역 변수 목록 확인 및 변수 삭제

- 전역 변수 목록을 확인합니다.

```
>>> %whos
```

- 전역 변수 목록에서 데이터프레임만 확인합니다.

```
>>> %whos DataFrame
```

- df\_na를 전역 변수 목록에서 삭제합니다.

```
>>> del df_na
```

- 전역 변수 목록에서 데이터프레임만 확인합니다.

```
>>> %whos DataFrame
```

# 날짜 인덱스 필터링

- df에서 2020년인 행을 선택합니다.(인덱싱)

```
>>> df.loc['2020'] # [참고] df의 인덱스가 날짜 인덱스이므로 'yyyy' 형태의 문자열을 지정해 해당 연도의 행을 선택할 수 있습니다.
```

- df에서 2020년 1월인 행을 선택합니다.(인덱싱)

```
>>> df.loc['2020-01'] # [참고] 'yyyy-mm' 형태의 문자열을 지정해 해당 월의 행을 선택할 수 있습니다.  
[주의] 팬시 인덱싱에서는 인덱스의 부분 문자열을 지원하지 않습니다.
```

- df에서 2020년 1월 2일, 3일인 행을 선택합니다.(팬시 인덱싱)

```
>>> df.loc[['2020-01-02', '2020-01-03']] # [주의] df에 없는 인덱스를 지정하면 에러를 일으킵니다.
```

- df에서 2020년 1월 1일부터 7일까지 연속된 행을 선택합니다.(슬라이싱)

```
>>> df.loc['2020-01-01':'2020-01-07'] # [참고] 슬라이싱에서는 인덱스의 부분 문자열을 지원합니다.  
>>> df.loc['2020-01':'2020-02']
```

## [참고] 날짜 인덱스의 속성 및 메서드 소개

| 구분                               | 상세 내용                                                                  |
|----------------------------------|------------------------------------------------------------------------|
| <code>df.index.year</code>       | <code>df.index</code> 의 원소에서 년(year)을 정수형 인덱스로 반환합니다.                  |
| <code>df.index.month</code>      | <code>df.index</code> 의 원소에서 월(month)을 정수형 인덱스로 반환합니다.                 |
| <code>df.index.day</code>        | <code>df.index</code> 의 원소에서 일(day)을 정수형 인덱스로 반환합니다.                   |
| <code>df.index.hour</code>       | <code>df.index</code> 의 원소에서 시(hour)을 정수형 인덱스로 반환합니다.                  |
| <code>df.index.minute</code>     | <code>df.index</code> 의 원소에서 분(minute)을 정수형 인덱스로 반환합니다.                |
| <code>df.index.second</code>     | <code>df.index</code> 의 원소에서 초(second)를 정수형 인덱스로 반환합니다.                |
| <code>df.index.dayofweek</code>  | <code>df.index</code> 의 원소에서 요일을 정수형 인덱스로 반환합니다.(Monday: 0, Sunday: 6) |
| <code>df.index.day_name()</code> | <code>df.index</code> 의 원소에서 요일을 문자열 인덱스로 반환합니다.(로케일에 따라 다름)           |
| <code>df.index.strftime()</code> | <code>df.index</code> 의 원소에서 지정한 날짜시간 포맷의 문자열 인덱스로 반환합니다.              |

## [참고] 날짜시간 주요 포맷

| 구분 | 상세 내용             | 예시         |
|----|-------------------|------------|
| %Y | 연도(4자리)           | '2025'     |
| %y | 연도(2자리)           | '25'       |
| %m | 월(정수, 01~12)      | '12'       |
| %B | 월(Locale 전체)      | 'December' |
| %b | 월(Locale 단축)      | 'Dec'      |
| %d | 일(정수, 01~31)      | '31'       |
| %j | 연중 일의 순번(001~366) | '365'      |
| %H | 시(24시간, 00~23)    | '01'       |
| %I | 시(12시간, 01~12)    | '01'       |

| 구분 | 상세 내용                       | 예시           |
|----|-----------------------------|--------------|
| %p | AM/PM 표기                    | 'AM'         |
| %M | 분(정수, 00~59)                | '02'         |
| %S | 초(정수, 00~59)                | '03'         |
| %f | 백만 분의 1초                    | '456789'     |
| %s | 유닉스 시간 <sup>UNIX time</sup> | '1703952123' |
| %W | 연중 주의 순번(00~52)             | '52'         |
| %w | 주중 요일의 순번(0~6)              | '0'          |
| %A | 요일(Locale 전체)               | 'Sunday'     |
| %a | 요일(Locale 단축)               | 'Sun'        |

## [참고] DatetimeIndex의 Frequency 문자열

| 구분       | 상세 내용                | 구분     | 상세 내용                |
|----------|----------------------|--------|----------------------|
| B        | 영업일(월요일~금요일) 단위      | W-MON  | 매주 월요일(요일 앵커 사용)     |
| D        | 일 단위                 | YE-JAN | 매년 1월 말일(월 앵커 사용)    |
| W        | 주 단위(일요일 기준 주간 집계)   | h      | 매시 단위                |
| ME, MS   | 매월 말일, 매월 시작일        | bh     | 영업일 기준 매시 단위(09~16시) |
| BME, BMS | 영업일 기준 매월 말일, 매월 시작일 | min    | 매분 단위                |
| QE, QS   | 분기 말일, 분기 시작일        | s      | 매초 단위                |
| BQE, BQS | 영업일 기준 분기 말일, 분기 시작일 | ms     | 천 분의 1초 단위           |
| YE, YS   | 연도 말일, 연도 시작일        | us     | 백만 분의 1초 단위          |
| BYE, BYS | 영업일 기준 연도 말일, 연도 시작일 | ns     | 십억 분의 1초 단위          |

## 날짜 인덱스의 속성 활용

- df에서 매년 1월인 행을 선택합니다.(불리언 인덱싱)

```
>>> df.loc[df.index.month == 1]
```

- df에서 매년 1월 2일인 행을 선택합니다.

```
>>> df.loc[(df.index.month == 1) & (df.index.day == 2)]
```

# [참고] 비트 연산자가 비교 연산자보다  
우선순위가 높으므로 비교 연산 코드를  
소괄호로 묶어주어야 합니다.

- df의 인덱스에서 정수 요일을 반환합니다.

```
>>> df.index.dayofweek # [참고] 정수 0은 Monday, 정수 6은 Sunday입니다.
```

- df에서 매주 월요일인 행을 선택합니다.

```
>>> df.loc[df.index.dayofweek == 0]
```

## 날짜 인덱스의 속성 활용(계속)

- df의 인덱스에서 영문 요일 문자열을 반환합니다.

```
>>> df.index.strftime(date_format='%A') # [참고] date_format 매개변수에 지정한 날짜 포맷에 맞는  
문자열을 반환합니다.
```

- df의 인덱스에서 영문 요일 문자열을 반환합니다.

```
>>> df.index.day_name() # [참고] locale 매개변수에 'ko_KR'을 지정하면 요일을 한글 문자열로 반환합니다.  
[주의] Colab 환경에서는 한국 로케일을 바로 사용할 수 없습니다.
```

- 한글 요일 문자열을 매핑할 딕셔너리를 생성합니다.

```
>>> weekday_kor = {0: '월', 1: '화', 2: '수', 3: '목', 4: '금', 5: '토', 6: '일'}
```

- df의 인덱스에서 정수 요일을 한글 요일명으로 매핑합니다.

```
>>> df.index.dayofweek.map(mapper=weekday_kor)
```

## [참고] 집계 함수 활용법

- df에서 매년 첫 번째 거래일인 행을 선택합니다.

```
>>> df.groupby(by=df.index.year).first() # [참고] first 메서드는 그룹 집계 함수이므로, 그룹 키를  
# 새로운 인덱스로 설정합니다.
```

- df에서 매년 마지막 거래일인 행을 선택합니다.

```
>>> df.groupby(by=df.index.year).last()
```

- df에서 매년 첫 번째 거래일인 행을 선택합니다.

```
>>> df.groupby(by=df.index.year).nth(n=0) # [참고] nth 메서드는 n번째 행을 선택하는 함수이므로,  
# 원본 인덱스를 유지합니다.
```

- df에서 매년 마지막 거래일인 행을 선택합니다.

```
>>> df.groupby(by=df.index.year).nth(n=-1)
```

# 주식 데이터 분석 지표 계산

수익률, 변동성, 이동 평균

# 주식 데이터 분석 지표

| 구분  | 지표        | 정의                  | 특징                                                    |
|-----|-----------|---------------------|-------------------------------------------------------|
| 수익률 | 단순 수익률    | 전일 대비 종가 변화율        | 단순 수익률은 가격의 상대적 변화를 직관적으로 나타내며, 가장 기본적인 성과 측정 지표입니다.  |
| 수익률 | 로그 수익률    | 종가 비율의 자연로그 값       | 로그 수익률은 시계열 모델링과 통계 분석에 적합하며, 수학적으로 다루기 쉬운 수익률 지표입니다. |
| 변동성 | 이동 표준편차   | 일정 기간 수익률의 표준편차     | 변동성은 가격의 불확실성과 리스크 수준을 나타내며, 수익률의 분산 정도를 기반으로 계산됩니다.  |
| 추세  | 이동 평균     | 일정 기간 종가의 평균        | 이동 평균은 가격의 잡음을 제거하고, 상승/하락 추세를 직관적으로 보여줍니다.           |
| 추세  | 모멘텀       | 과거 대비 가격 변화량 또는 변화율 | 모멘텀은 가격이 상승하거나 하락하는 힘의 방향 및 강도를 나타냅니다.(투자 의사결정 활용 지표) |
| 거래량 | 거래량 이동 평균 | 일정 기간 거래량의 평균       | 거래량은 가격 움직임의 신뢰도를 판단하고, 시장 참여 강도를 나타내는 보조 지표입니다.      |

## [참고] 단순 수익률 vs 로그 수익률

| 구분        | 단순 수익률                                         | 로그 수익률                                                      |
|-----------|------------------------------------------------|-------------------------------------------------------------|
| 정의        | 전일 대비 종가 변화율( $-1 \sim \infty$ )               | 종가 비율에 로그를 취한 값( $-\infty \sim \infty$ )                    |
| 수식        | $R_t = \frac{P_t}{P_{t-1}} - 1$                | $r_t = \ln \left( \frac{P_t}{P_{t-1}} \right)$              |
| 직관성       | 매우 직관적                                         | 직관성은 다소 떨어짐                                                 |
| 해석        | 전일 대비 가격이 몇 % 올랐는가                             | 연속 복리 기준 변화량                                                |
| 누적 수익률    | 곱셈 필요                                          | 덧셈으로 충분                                                     |
| 누적 수익률 수식 | $R_{\text{cum}} = \prod_{t=1}^T (1 + R_t) - 1$ | $R_{\text{cum}} = \exp \left( \sum_{t=1}^T r_t \right) - 1$ |
| 활용 분야     | 실무 보고, 투자 성과 설명                                | 금융공학, 시계열 모형, 수리적 분석                                        |

# [참고] 일부 전통적 금융모형은 수익률의 정규성 가정을 활용합니다.

로그 수익률을 정규분포로 가정하면, 주가는 로그정규분포를 따르는 모형으로 표현할 수 있습니다.

# 일간 수익률 계산

- 단순 수익률을 계산하고 df에 추가합니다.

```
>>> df['Simple_Return'] = df['Close'].pct_change()
```

*# [참고] pct\_change 메서드는 직전 값 대비 현재 값의 변화율을 반환합니다.*

- 로그 수익률을 계산하고 df에 추가합니다.

```
>>> df['Log_Return'] = np.log(df['Close'] / df['Close'].shift(periods=1))
```

*# [참고] shift 메서드는 periods 매개변수에 지정한 정수만큼 위(음수) 또는 아래(양수)로 이동시킵니다.*

- df에서 열별 결측값 개수를 확인합니다.

```
>>> df.isna().sum()
```

# 누적 수익률 계산

- 누적 수익률을 계산하고 df에 추가합니다.

```
>>> df['Cum_Return'] = (df['Simple_Return'] + 1).cumprod() - 1
```

# [참고] 단순 수익률에 1을 더해  
원금을 1로 설정합니다.

- df의 처음 10행을 확인합니다.

```
>>> df.head(n=10)
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

- 로그 수익률을 누적한 뒤 단순 누적 수익률로 변환합니다.

```
>>> np.exp(df['Log_Return'].cumsum()) - 1
```

# [참고] 로그 누적 수익률을 지수 변환한 뒤, 1을 차감하면  
단순 누적 수익률과 동일한 값이 됩니다.

## 누적 수익률 백분율 계산

- 누적 수익률의 백분율을 계산하고 df에 추가합니다.

```
>>> df['Cum_Return_Pct'] = df['Cum_Return'] * 100
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

- 시작 종가와 마지막 종가를 각각 변수에 할당합니다.

```
>>> base_close, last_close = df['Close'].iloc[[0, -1]]
```

- 전체 기간 누적 수익률을 확인합니다.

```
>>> (last_close / base_close - 1) * 100
```

# 변동성의 이해

- 변동성은 주가 수익률이 얼마나 크게 변하는지를 나타내는 지표입니다.
  - 주가가 얼마나 크게 오르고 내리는지를 수치로 표현한 것입니다.
- 변동성이 크다는 것은 가격 변동 폭이 커서 투자 위험이 크다는 것을 의미합니다.
  - 반대로 변동성이 작으면 가격이 비교적 안정적으로 움직인다고 볼 수 있습니다.
- 변동성은 일반적으로 수익률의 표준편차로 계산합니다.
  - 수익률이 평균을 중심으로 얼마나 퍼져 있는지를 측정하는 방식입니다.
  - 변동성은 가격 움직임의 강도(크기)를 나타내는 지표라고 이해할 수 있습니다.

# 변동성의 한계와 동적 변동성

- 전체 기간의 수익률을 기준으로 계산하는 변동성은 하나의 평균적인 값입니다.
  - 특정 시점에서 발생한 시장 변화나 이벤트를 충분히 반영하지 못하는 한계가 있습니다.
  - 금융 시장에서는 변동성이 시간에 따라 지속적으로 변화하는 특징을 보입니다.
- 변동성의 한계를 보완하기 위해 이동 변동성을 사용합니다.
  - 일정 기간 동안의 수익률을 기준으로 변동성을 계산하는 방식입니다.
  - 이동 변동성은 최근 시장 상태를 반영할 수 있으며, 시장이 안정적인지 혹은 불안정한지 여부를 시계열로 확인할 수 있습니다.
  - 일반적으로 20 거래일(약 1개월) 기준의 이동 표준편차를 많이 사용합니다.

## 변동성 계산

- 단순 수익률의 평균을 확인합니다.

```
>>> df['Simple_Return'].mean()
```

- 단순 수익률의 표준편차를 확인합니다.

```
>>> df['Simple_Return'].std()
```

- 단순 수익률의 20일 이동 표준편차를 계산하고 df에 추가합니다.

```
>>> df['Volatility_20'] = df['Simple_Return'].rolling(window=20).std()
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

# 이동 평균

- 이동 평균은 일정 기간 동안의 가격 평균이며, 가격의 흐름을 파악하는 지표입니다.
  - 이동 평균은 가격 데이터를 부드럽게 만들어 전반적인 추세를 파악하는 데 사용됩니다.
- 이동 평균을 활용하면 다음과 같이 해석할 수 있고, 매수/매도 규칙을 만들 수 있습니다.
  - 단기(20일) 이동 평균: 최근 가격 흐름을 반영하며, 가격 변화에 빠르게 반응합니다.
  - 중기(60일) 이동 평균: 전체적인 추세를 반영하며, 부드러운 흐름을 보입니다.

| 구분                  | 해석                |
|---------------------|-------------------|
| 단기 이동 평균 > 중기 이동 평균 | 상승 추세 → 매수 신호로 활용 |
| 단기 이동 평균 < 중기 이동 평균 | 하락 추세 → 매도 신호로 활용 |

# [주의] 실제 투자에서는 거래량과 변동성 등 추가적인 조건을 함께 고려합니다.

# 이동 평균 계산

- 종가의 20일 이동 평균을 계산하고 df에 추가합니다.

```
>>> df['MA_20'] = df['Close'].rolling(window=20).mean()
```

- 종가의 60일 이동 평균을 계산하고 df에 추가합니다.

```
>>> df['MA_60'] = df['Close'].rolling(window=60).mean()
```

- df의 처음 10행을 확인합니다.

```
>>> df.head(n=10)
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

# 주식 데이터 시각화

# 데이터 시각화 개요

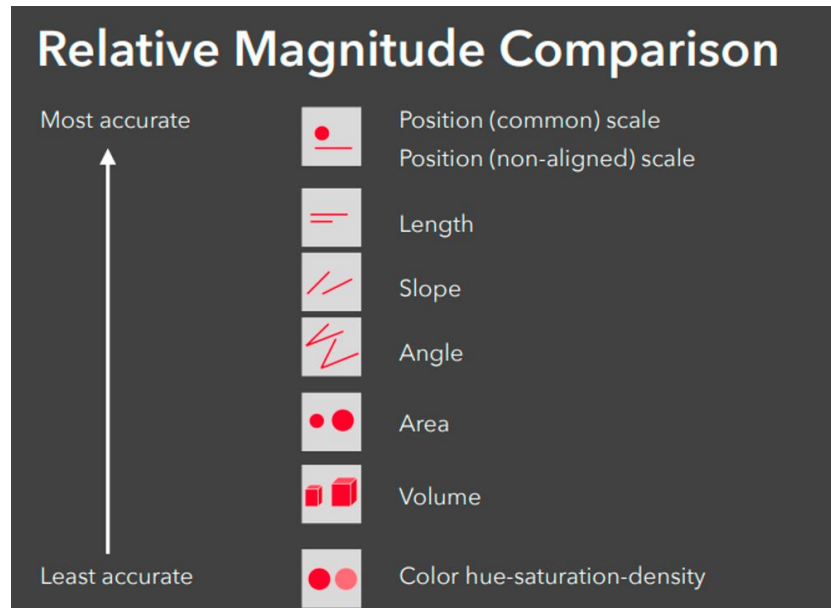
- 데이터 시각화는 단순히 데이터를 차트로 표현하는 기술이 아니라, 데이터를 이해하고 정보를 다른 사람에게 명확하게 전달하는 커뮤니케이션 도구입니다.
  - 데이터 시각화는 수치형 데이터를 시각적 언어로 번역하는 행위입니다.
  - 그 목적은 데이터에 담긴 정보를 쉽고, 빠르고, 정확하게 전달하는 것입니다.
- 사람은 정보를 인지할 때, 오감 중에서 시각이 가장 큰 비중을 차지하므로 데이터 시각화는 효율적인 커뮤니케이션의 핵심 수단이라고 할 수 있습니다.
  - 데이터 분석 결과를 시각적으로 표현하면, 데이터를 직접 분석하지 않은 사람도 빠르게 이해할 수 있습니다.
  - 상대방의 판단과 행동을 변화시키는 것이 데이터 시각화의 궁극적인 목적입니다.

# 데이터 리터러시와 시각화의 관계

- 데이터 리터러시는 데이터를 읽고, 이해하고, 해석하는 능력입니다.
  - 수치형 데이터 속에 담긴 객관적인 사실을 올바르게 읽어야 합니다.
  - 비교를 통해 의미를 해석해야만 '좋다'와 '나쁘다'를 판단할 수 있습니다.
    - 대표적인 비교는 개인 vs 집단(횡단면 비교), 현재 vs 과거(시계열 비교)입니다.
    - 평균, 분산, 범위 등의 기술통계량은 비교를 위한 도구입니다.
- 데이터 시각화는 데이터에서 발견한 정보를 시각적으로 전달하는 행위입니다.
  - 데이터를 읽는 능력과 데이터를 시각적으로 표현하는 능력은 서로 보완적인 관계입니다.
  - 데이터 시각화는 데이터 리터러시를 완성하는 커뮤니케이션의 기술입니다.

# 인간의 인지적 특성과 시각 요소

- 인간은 정보를 인지할 때 오감 중 시각에 의존하는 경향이 가장 높습니다.
  - 인지심리학 및 뇌과학 연구에 따르면, 인간은 복합 감각 자극 중 시각 정보에 본능적으로 최우선 순위를 부여합니다.
- 오른쪽 표는 인간이 그래프를 해석할 때 시각 요소별 정확도 순서를 나타냅니다.
  - 위치와 길이 기반의 그래프(막대, 선, 산점도)가 데이터 비교에 적합합니다.



# 출처: <https://community.heartcount.io/en/bar-chart/>

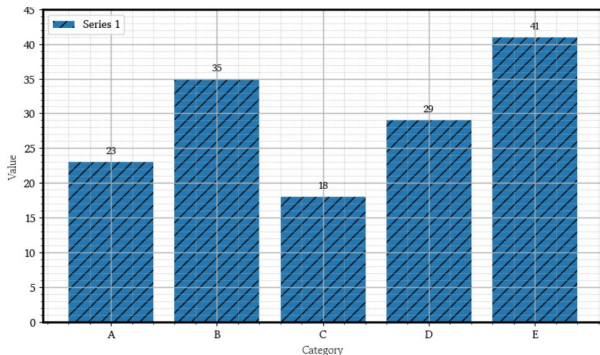
# 좋은 시각화의 조건: Data-Ink Ratio

- 좋은 시각화란 불필요한 장식을 최소화하고, 데이터의 본질에 집중하는 것입니다.

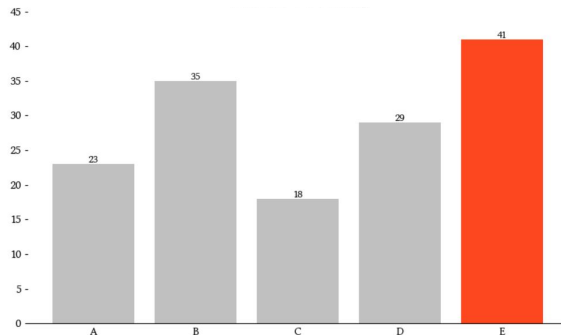
Edward Tufte(1983)는 이러한 원리를 체계화하여 Data-Ink Ratio라는 개념을 제시했습니다.

$$\text{Data-Ink Ratio} = \frac{\text{Data Ink}}{\text{Total Ink}} = \frac{\text{Signal}}{\text{Signal} + \text{Noise}}$$

# [참고] 값이 클수록 데이터 자체에 더 많은 비중을 둔 효율적인 시각화임을 의미합니다.



[장식이 많은 차트]



[불필요한 장식을 제거한 차트]

# [참고] 색이름 목록

- matplotlib에서 제공하는 148가지 색이름과 Hex Code를 딕셔너리로 확인합니다.

```
>>> from matplotlib import colors
```

```
>>> colors.CSS4_COLORS
```

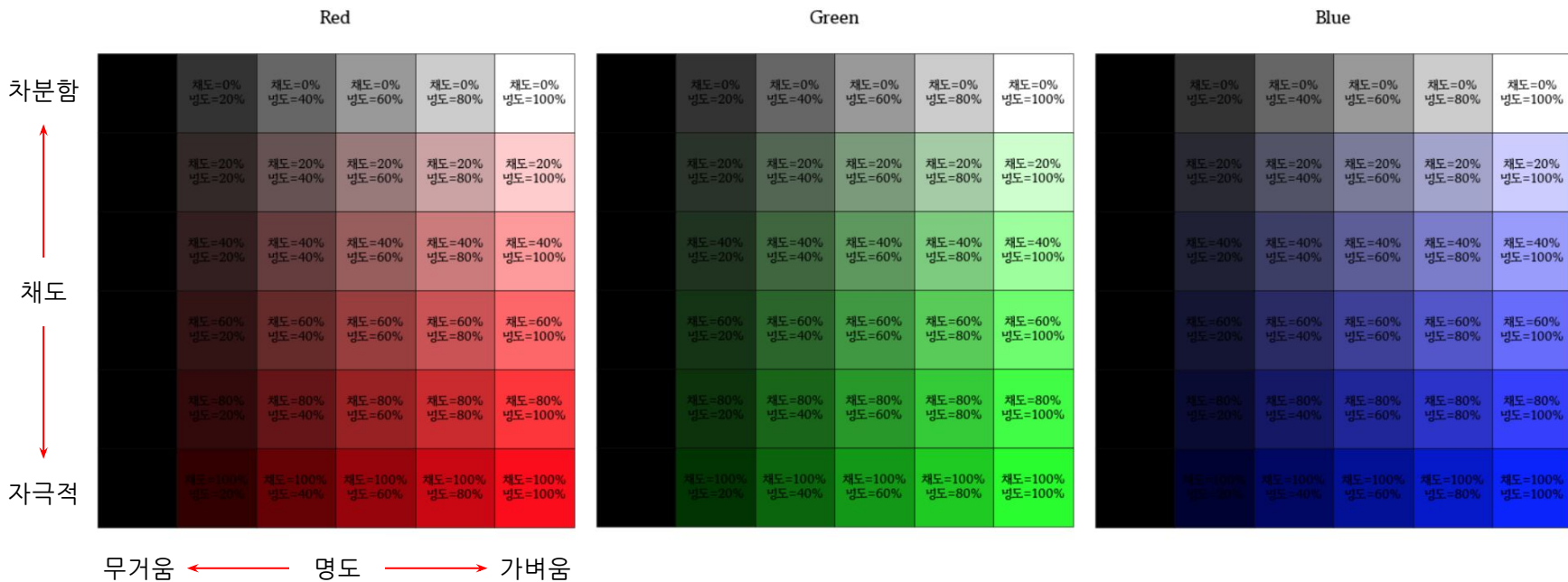
|            |                |                   |                 |             |                      |                |                 |
|------------|----------------|-------------------|-----------------|-------------|----------------------|----------------|-----------------|
| black      | bisque         | forestgreen       | slategrey       | firebrick   | khaki                | darkslategray  | darkorchid      |
| dimgray    | darkorange     | limegreen         | lightsteelblue  | maroon      | palegoldenrod        | darkslategray  | darkviolet      |
| dimgray    | darkgray       | darkgreen         | cornflowerblue  | darkred     | darkkhaki            | teal           | mediumorchid    |
| gray       | antiquewhite   | green             | royalblue       | red         | ivory                | darkcyan       | thistle         |
| gray       | tan            | lime              | ghostwhite      | mistyrose   | beige                | aqua           | plum            |
| darkgray   | navajowhite    | seagreen          | lavender        | salmon      | lightyellow          | cyan           | violet          |
| darkgray   | blanchedalmond | mediumseagreen    | midnightblue    | tomato      | lightgoldenrodyellow | darkturquoise  | purple          |
| silver     | papayawhip     | springgreen       | navy            | darksalmon  | olive                | cadetblue      | darkmagenta     |
| lightgray  | moccasin       | mintcream         | darkblue        | coral       | yellow               | powderblue     | fuchsia         |
| lightgray  | orange         | mediumspringgreen | mediumblue      | orangered   | olivedrab            | lightblue      | magenta         |
| gainsboro  | wheat          | mediumaquamarine  | blue            | lightsalmon | yellowgreen          | deepskyblue    | orchid          |
| whitesmoke | oldlace        | aquamarine        | slateblue       | sienna      | darkolivegreen       | skyblue        | mediumvioletred |
| white      | floralwhite    | turquoise         | darkslateblue   | seashell    | greenyellow          | lightskyblue   | deeppink        |
| snow       | darkgoldenrod  | lightseagreen     | mediumslateblue | chocolate   | chartreuse           | steelblue      | hotpink         |
| rosybrown  | goldenrod      | mediumturquoise   | mediumpurple    | saddlebrown | lawngreen            | aliceblue      | lavenderblush   |
| lightcoral | cornsilk       | azure             | rebeccapurple   | sandybrown  | honeydew             | dodgerblue     | palevioletred   |
| indianred  | gold           | lightcyan         | blueviolet      | peachpuff   | darkseagreen         | lightslategray | crimson         |
| brown      | lemonchiffon   | paleturquoise     | indigo          | peru        | palegreen            | lightslategray | pink            |
|            |                |                   |                 | linen       | lightgreen           | slategray      | lightpink       |

# [출처] [https://matplotlib.org/stable/gallery/color/named\\_colors.html](https://matplotlib.org/stable/gallery/color/named_colors.html)

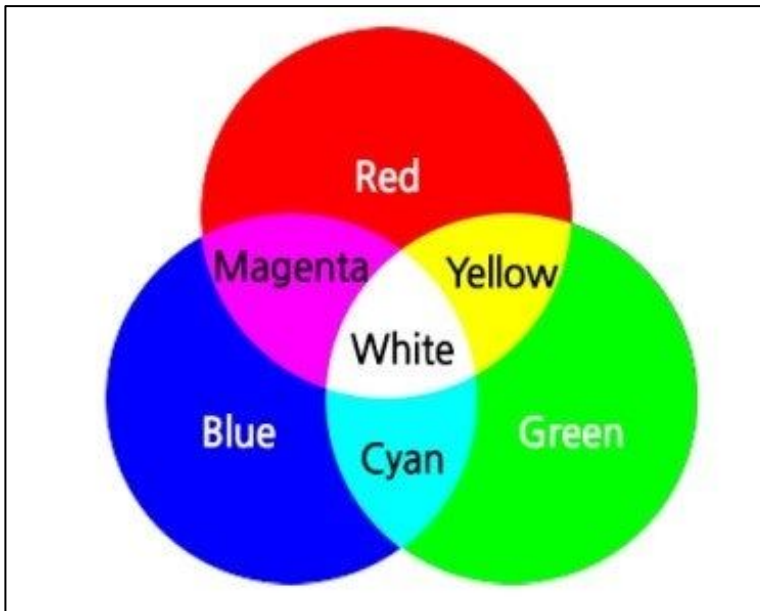
## [참고] 색의 기본 구성 요소

| 구분                 | 정의                                   | 인지적 효과                                                                                                                                                           | 활용 방법     |
|--------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 색상<br>(hue)        | 빨강, 초록, 파랑 등 색 자체가 갖는 고유의 특성을 의미합니다. | <ul style="list-style-type: none"> <li>- 색상은 상징적인 의미를 지닙니다.</li> <li>- 빨강: 열정, 사랑, 위험, 경고</li> <li>- 초록: 안정, 균형, 성장, 발전</li> <li>- 파랑: 신뢰, 차분, 이성, 냉정</li> </ul> | 범주형 구분    |
| 명도<br>(Brightness) | 색의 밝고 어두운 정도입니다.                     | <ul style="list-style-type: none"> <li>- 명도는 시각적 무게감을 전달합니다.</li> <li>- 밝을수록 가볍고, 적은 느낌</li> <li>- 어두울수록 무겁고, 많은 느낌</li> <li>- 명도가 0%이면 검정색이 됩니다.</li> </ul>     | 연속형 표현    |
| 채도<br>(Saturation) | 색의 맑고 탁한 정도입니다.                      | <ul style="list-style-type: none"> <li>- 채도는 강도와 분위기를 결정합니다.</li> <li>- 높을수록 자극적이고 주의 집중 효과</li> <li>- 낮을수록 차분하고 배경 효과</li> <li>- 채도가 0%이면 무채색이 됩니다.</li> </ul>  | 강조 항목에 사용 |

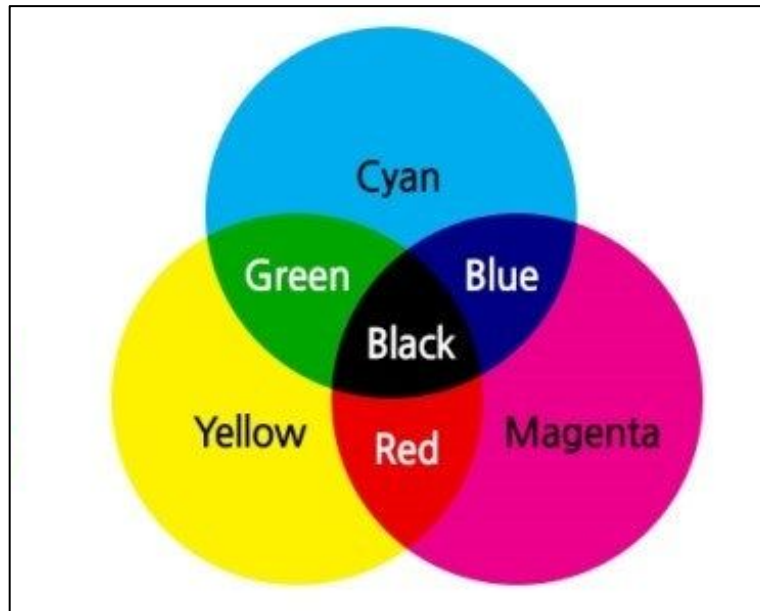
# [참고] 빛의 삼원색의 명도와 채도



## [참고] 빛의 삼원색 vs 색의 삼원색



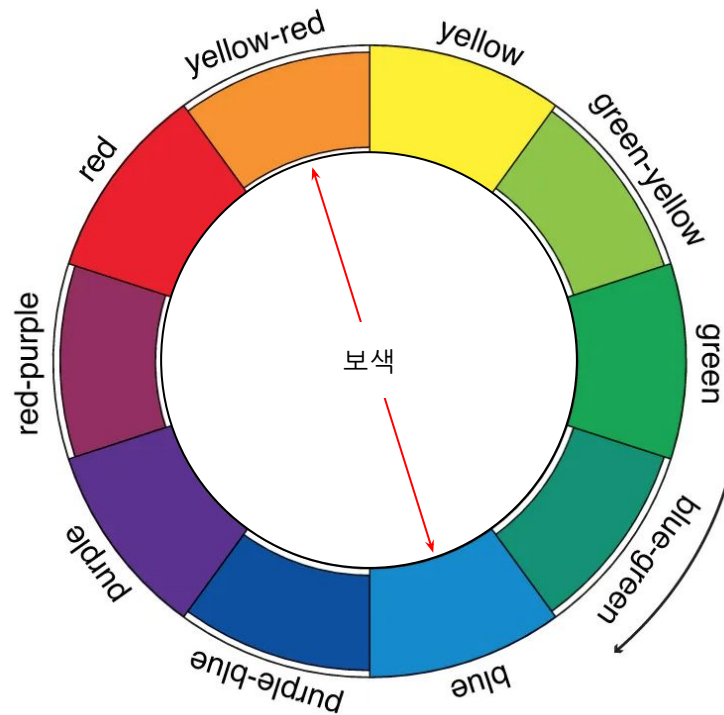
[빛의 삼원색]



[색의 삼원색]

## [참고] Munsell Color Wheel

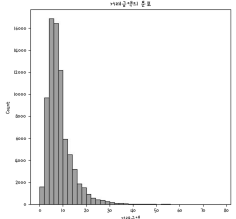
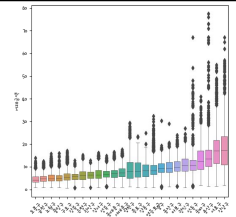
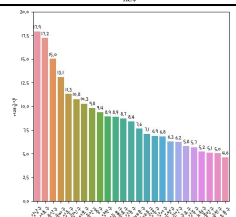
- 먼셀 색상환은 인간의 지각 균등성을 고려하여 설계된 색 체계입니다.
  - 색상환에서 가장 멀리 떨어진 두 색은 서로의 보색입니다.
  - 보색은 단순히  $180^\circ$  반대색이 아니라, 시각적으로 균형을 이루는 조합입니다.
  - 보색이 나란히 있을 때 두 색은 서로를 더욱 선명하게 보이게 합니다.
  - 파랑과 주황은 정반대의 감정을 주면서 동시에 안정감 있는 조화를 이룹니다.



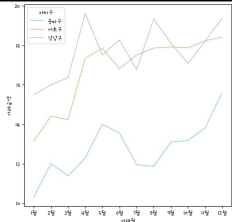
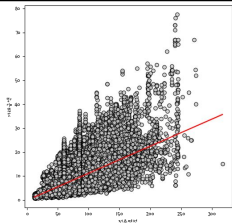
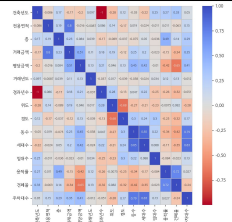
## [참고] 좋은 시각화를 위한 색 사용 전략

- 다음은 효과적인 색 사용을 위한 일곱 가지 핵심 전략입니다.
  - 색은 메시지를 전달하는 주요 대상에만 제한적으로 사용합니다.
  - 하나의 색에는 하나의 의미만 부여합니다.
  - 색이 가진 상징적 의미를 의식합니다. 빨강은 자극, 파랑은 안정의 의미로 인식됩니다.
  - 색상 수는 5개 미만으로 최소화합니다. 과도한 색상은 정보 해석을 방해합니다.
  - 기본 요소에는 낮은 채도의 색을, 강조할 요소에는 높은 채도의 색을 사용합니다.
  - 보색을 활용해 시각적인 균형을 유지합니다.
  - 의미 없는 배경색은 사용하지 않습니다. Data-Ink Ratio를 높이는 것이 좋습니다.

# 데이터 시각화 종류

| 구분     | 예시                                                                                | 특징                                                                                                                                                                                                             |
|--------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 히스토그램  |  | <ul style="list-style-type: none"> <li>- 히스토그램은 일변량 연속형 변수의 도수분포표를 시각화한 그래프입니다.</li> <li>- 세로축의 기본값은 개수(도수)이며 확률밀도로 변경할 수 있습니다.</li> <li>- 히스토그램의 막대는 서로 붙어 있으며 세로축을 밀도로 변경하면 막대의 총면적은 확률 1을 의미합니다.</li> </ul> |
| 상자 그림  |  | <ul style="list-style-type: none"> <li>- 일변량 연속형 변수의 분포에 사분위수와 이상치를 추가한 그래프입니다.</li> <li>- 가로축에 범주형 변수를 지정하면 집단 간 연속형 변수의 분포를 비교할 수 있습니다.</li> </ul>                                                           |
| 막대 그래프 |  | <ul style="list-style-type: none"> <li>- 일변량 막대 그래프는 범주형 변수의 도수를 막대로 그린 그래프입니다.</li> <li>- 이변량 막대 그래프는 범주형 변수에 따라 연속형 변수의 크기를 비교할 수 있습니다.</li> </ul>                                                           |

# 데이터 시각화 종류(계속)

| 구분    | 예시                                                                                 | 특징                                                                                                                                                                           |
|-------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 선 그래프 |   | <ul style="list-style-type: none"> <li>- 선 그래프는 시간에 따라 연속형 변수의 변화를 표현한 그래프입니다.</li> <li>- 주가 데이터와 같이 시계열 변수를 시각화할 때 사용합니다.</li> </ul>                                        |
| 산점도   |   | <ul style="list-style-type: none"> <li>- 산점도는 이변량 연속형 변수의 관계를 점으로 표현한 그래프입니다. 선형 관계, 군집 및 이상치 등을 확인할 수 있습니다.</li> <li>- 선형 회귀분석의 입력변수와 목표변수에 직선의 관계가 존재하는지 확인합니다.</li> </ul> |
| 히트맵   |  | <ul style="list-style-type: none"> <li>- 히트맵은 여러 변수 간 관계의 크기를 색의 분포로 표현한 그래프입니다.</li> <li>- 선형 회귀분석의 입력변수 간 상관관계가 크면 다중공선성 문제가 발생할 수 있으므로 히트맵을 통해 확인할 수 있습니다.</li> </ul>     |

# Colab에서 한글 폰트 설치

- 현재 Colab에 설치된 폰트 경로 목록을 확인합니다.

```
>>> !fc-list # [참고] fc-list는 현재 설치된 폰트의 파일 경로와 패밀리, 스타일 정보를 출력합니다.
```

- 설치된 전체 폰트의 패밀리 이름만 출력합니다.

```
>>> !fc-list ':' family # [참고] 콜론은 fontconfig의 쿼리 구문 시작 기호입니다.  
                        콜론 뒤에 family를 지정하면 폰트의 패밀리 이름만 출력합니다.
```

- Colab에서 사용할 나눔 폰트를 설치합니다.

```
>>> !apt-get install -y fonts-nanum # [참고] apt-get은 리눅스 저장소에 등록된 패키지를 의존성까지  
                                     포함하여 설치/업데이트/삭제하는 명령어입니다.
```

- 설치된 한글 폰트의 패밀리 이름을 중복 없이 오름차순 정렬하여 출력합니다.

```
>>> !fc-list ':lang=ko' family | sort | uniq # [참고] 콜론 오른쪽에 한글 폰트만 필터링하는 조건과  
                                             중복 제거 및 오름차순 정렬 옵션을 추가합니다.
```

# 한글 폰트명 탐색

- 필요한 모듈을 임포트합니다.

```
>>> import matplotlib.font_manager as fm # [참고] 컴퓨터에 설치된 폰트 정보를 확인할 때 사용합니다.
```

```
>>> import matplotlib.pyplot as plt
```

```
>>> import seaborn as sns
```

- 설치된 폰트 파일 경로를 리스트로 생성합니다.

```
>>> font_list = fm.findSystemFonts(fonttext='ttf')
```

- font\_list의 원소 개수를 확인합니다.

```
>>> len(font_list)
```

## 한글 폰트명 탐색(계속)

- font\_list에서 특정 폰트명을 포함하는 폰트 파일 경로를 선택하여 font\_path에 할당합니다.

```
>>> font_path = sorted([font for font in font_list if 'Nanum' in font])
```

- font\_path를 확인합니다.

```
>>> font_path
```

- 선택한 폰트 파일의 폰트명을 리스트로 확인합니다.

```
>>> [fm.FontProperties(fname=font).get_name() for font in font_path]
```

- matplotlib 라이브러리의 폰트 관리자 객체를 초기화합니다.

```
>>> fm.fontManager.__init__() # [참고] 컴퓨터에 설치된 폰트 파일들을 다시 스캔하여 내부 폰트 캐시를  
                             재구성하여 새로 설치한 한글 폰트를 사용할 수 있게 합니다.
```

# 그래프 요소 설정

- 한글 폰트와 글자 크기를 설정합니다. # [참고] rc는 runtime configuration의 줄임말로, pyplot을 실행하는 환경을 의미합니다.

```
>>> plt.rc(group='font', family='NanumBarunGothic', size=10)
```

- 그래프 크기와 해상도를 설정합니다.

```
>>> plt.rc(group='figure', figsize=(8, 4), dpi=120)
```

- 축에 유니코드 마이너스를 출력하지 않도록 설정합니다.

```
>>> plt.rc(group='axes', unicode_minus=False) # [참고] x축과 y축 눈금에서 음수 앞에 '-' 대신 '␣'를 출력하지 않도록 설정합니다.
```

- 범례에 채우기 색과 테두리 색을 추가합니다. # [참고] fc 매개변수에 채우기 색을, ec 매개변수에 테두리 색을 지정합니다. '0'은 검정, '1'은 흰색입니다.

```
>>> plt.rc(group='legend', frameon=True, fc='0.9', ec='0.9')
```

# 종가 추이 선 그래프 시각화

- 종가 추이를 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Close', color='0', lw=1)
```

```
>>> plt.axhline(y=base_close, color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='주가(원)')
```

```
>>> plt.title(label=f'{ticker} 종가(원)', fontweight='bold')
```

```
>>> plt.show()
```

# 이동 평균 선 그래프 시각화

- 종가와 이동 평균을 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Close', color='0.5', lw=1, label='종가')
```

```
>>> sns.lineplot(data=df, x=df.index, y='MA_20', color='blue', lw=1, label='단기 이평')
```

```
>>> sns.lineplot(data=df, x=df.index, y='MA_60', color='red', lw=1.5, label='중기 이평')
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='주가(원)')
```

```
>>> plt.title(label=f'{ticker} 종가 및 이동 평균', fontweight='bold')
```

```
>>> plt.show()
```

# 로그 수익률 선 그래프 시각화

- 로그 수익률의 변화를 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Log_Return', color='red', lw=1)
```

```
>>> plt.axhline(y=0, color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='로그 수익률')
```

```
>>> plt.title(label=f'{ticker} 로그 수익률', fontweight='bold')
```

```
>>> plt.show()
```

# [참고] 로그 수익률이 0보다 크면 전일 대비 주가가 상승하고, 0보다 작으면 하락한 것을 의미합니다.  
로그 수익률이 큰 폭으로 움직이는 구간을 통해 주가의 급등락 시점을 확인할 수 있습니다.

# 누적 수익률 선 그래프 시각화

- 누적 수익률 추이를 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Cum_Return_Pct', color='red', lw=1)
```

```
>>> plt.axhline(y=0, color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='백분율(%))')
```

```
>>> plt.title(label=f'{ticker} 누적 수익률 백분율(%)', fontweight='bold')
```

```
>>> plt.show()
```

# 로그 수익률 히스토그램 시각화

- 로그 수익률의 기술통계량을 확인합니다.

```
>>> df['Log_Return'].describe()
```

- 로그 수익률의 분포를 히스토그램과 KDE로 시각화합니다. # [참고] 로그 수익률은 정규분포처럼 보이지만 실제로는 양쪽 꼬리가 정규분포보다 두껍다는 특징을 가집니다.

```
>>> sns.histplot(data=df, x='Log_Return', fc='0.8', ec='1',  

                 binrange=(-0.15, 0.15), bins=60, stat='density')
```

```
>>> sns.kdeplot(data=df, x='Log_Return', color='red', lw=1.5)
```

```
>>> plt.title(label=f'{ticker} 로그 수익률 히스토그램', fontweight='bold')
```

```
>>> plt.show()
```

# 로그 수익률 Normal Q-Q Plot 시각화

- 필요한 모듈을 임포트합니다.

```
>>> from scipy import stats
```

- 로그 수익률을 Normal Q-Q Plot으로 시각화합니다. *# [참고] 점들이 기준 직선에 대체로 가까우면 데이터가 정규분포에 가까운 것으로 판단합니다.*

```
>>> stats.probplot(x=df['Log_Return'].dropna(), dist='norm', plot=plt)
```

```
>>> plt.xlabel(xlabel='정규분포 기준 이론적 분위수')
```

```
>>> plt.ylabel(ylabel='표본 데이터 분위수')
```

```
>>> plt.title(label=f'{ticker} Normal Q-Q Plot', fontweight='bold')
```

```
>>> plt.show()
```

## 20일 이동 변동성 선 그래프 시각화

- 20일 이동 변동성을 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Volatility_20', color='red', lw=1)
```

```
>>> plt.axhline(y=df['Volatility_20'].mean(), color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='표준편차')
```

```
>>> plt.title(label=f'{ticker} 20일 이동 표준편차', fontweight='bold')
```

```
>>> plt.show()
```

## 종목 간 성과 비교

- 지금까지 주식 데이터를 수집하고, 전처리 및 분석 지표를 계산하여 시각화까지 일련의 분석 과정을 실습했습니다.
- 미국 주식과 국내 주식 종목을 참고하여 같은 기간 가장 높은 누적 수익률을 기록한 종목을 탐색하는 실습을 진행하겠습니다.
  - 같은 기간 동안 각 종목의 누적 수익률을 계산하고, 상위 3개 종목을 선택하세요.
  - 단, 전체 기간에서 일간 수익률의 표준편차가 상위 30%에 해당하는 종목은 제외합니다.

| 미국 주식 티커                                                                                               | 국내 주식 티커                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'AAPL', 'MSFT', 'NVDA', 'GOOGL',<br>'META', 'TSLA', 'AMZN', 'NFLX',<br>'AMD', 'JPM', 'BA', 'XOM', 'V', | '005930.KS', '000660.KS', '035420.KS', '035720.KS',<br>'373220.KS', '051910.KS', '005380.KS', '000270.KS',<br>'207940.KS', '068270.KS', '105560.KS', '055550.KS', |

# End of Document